NIL Investment Analysis Report

Plan

Business Objective

College Basketball programs are facing more financial pressire to allocate NIL resources effectively. Programs have to balance the financial resources available for player compensation with goal of getting a competitive roster from it. Understanding hpw NIL spending relates to projected team performance can help programs evaluate their financial position relative to other schools in their conference.

Our goal is to estimate team NIL as a function of BPR and conference affiliation. The model will generate an NIL benchmark for each team, allowing us to compare a program's estimated NIL with the level predicted for a team with similar characteristics

Schools with estimated NIL below their benchamrk may want to do further looks as pontentially cost-efficient programs, while those above the benchmark may want to do additional review of their resource allocation

Because the analysis relies on estimated NIL values and projected rather than realized basketball performance, these comparisons are descriptive rather than causal. A difference from the model benchmark does not, by itself, establish whether a program is overpaying or underpaying its players.

Decision Problem and Objective Function

Our decision problem is to establish a reasonable NIL amount for each Division I basketball program based on its team strength and conference affiliation. These benchmarks can then support the amount of teams whose estimated NIL levels differ susbstantially from comparable programs.

Let Y_i represent the estimated NIL spending of team i, measured in millions of dollars. Let X_i contain the team's BPR and conference. The action a_i is the NIL benchmark produced by the model for that team.

We ended up using quadratic loss:

$$\ell(a_i, Y_i) = (Y_i - a_i)^2$$

Quadratic Loss penalizes larger prediction errors more heavily then smaller errors, making it appropriate for constructing benchmarks where lots of money discrepancies are big consequences

Our objective is to have a function that minimizes that expected loss

$$f^* = \arg\min_f E[(Y - f(X))^2]$$

Under squared-error loss, the best prediction for a given set of characteristics is the mean:

$$f^*(X_i) = E[Y_i \mid X_i]$$

Because we dont know the true relationship, we estimate using regression:

$$\widehat{R}(f) = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} (Y_i - \widehat{Y_i})^2$$

After our model, we calcuate each team's estimated NIL gap:

$$G_i = \widehat{Y_i} - Y_i$$

A positive gap means that estimated NIL is below the model's benchmark, while a negative gap means estimated NIL is above the benchmark

These gaps are indicator for teams that should have further investigation rather definitive measures of financial efficiency. The available data do not establish the causal effect of NIL spending on basketball performance or determine an optimal roster budget. Furthermore, the observed outcome is a third-party NIL estimate rather than verified actual expenditure.

Here is an example:

```
# Quadratic loss for NIL predictions in millions of dollars
def nil_squared_loss(actual_nil, predicted_nil):
    return (actual_nil - predicted_nil) ** 2


# Illustrative prediction: $2.5M benchmark versus $3.0M estimate
nil_squared_loss(
```

```
    actual_nil = 3.0,
    predicted_nil = 2.5
)
```

```
0.25
```

Build

Ideal Data and Data-Generating Process

Ideally, this model would use legit verified team NIL values alongside the independently measures BPR for all Division I men's basketball programs. Things like player compensation, recruiting outcomes, and historical team performance would help improve the relationship between spending and basketball strength from other factors that influence NIL valuations

For our current objective, the variables of interest are team NIL values (estimated), Bayesian Perfomrance Rating (BPR), and conference affiliation. We expect NIL expenditure and BPR to be postively associated because stronger basketball programs tend to have access to more money and more resources, although conference affiliation and other factors may affect this relationship.

Our data we used was Sideline's estimated team NIL values and EvanMiya's team and player BPR.

Model Specification

We used a multiple linear regression model in which the natural log of team NIL is predicted by BPR and conference affiliation:

$$\log(Y_i) = \beta_0 + \beta_1 BPR_i + \sum_{c \neq \text{BigTen}} \gamma_c I(C_i = c) + \epsilon_i$$

where:

$$\epsilon_i \overset{iid}{\sim} N(0, \sigma^2)$$

NIL was log-transformaed becuase NIL estimates have a very right skew. This transformation allows the relationship between BPR and NIl to be seen on a log scale.

The coefficient β_1 represents the expected change in log NIL associated with a +1 increase in BPR, holding all other variables constant. Its exact interpretation is $100(e^{\beta_1} - 1)$.

Conference was represented using indicator variables, with the Big Ten as the reference conference. These indicators allow teams from different conferences to have different intercepts while assuming a common BPR slope

The initial model assumes linearity on the log scale, independent errors, constant error variance, and normally distributed errors. It also assumes that the effect of BPR does not vary by conference. These simplifying assumptions will be evaluated using the observed data.

Under these distributional assumptions, the model-implied conditional mean NIL on the original dollar scale is:

$$E[Y_i \mid X_i] = e^{\mu_i + \sigma^2/2}$$

where μ_i is the predicted log NIL. This distinction matters because simply exponentiating μ_i produces the conditional median under the log-normal model, rather than its conditional mean.

Simulating Data and Recovering Parameters

```
import numpy as np
import polars as pl
import statsmodels.formula.api as smf
from plotnine import *

# Known parameters for the simulated data
beta_0 = 1.40
beta_bpr = 0.030
sigma = 0.30

conference_effects = {
    'BigTen': 0.00,
    'BigEast': -0.25,
    'WCC': -0.65,
    'Summit': -1.00
}

# Function to simulate team NIL data
def simulate_nil(n, beta_0, beta_bpr, conference_effects, sigma):
    rng = np.random.default_rng(42)

    # Assign conferences and generate correlated BPR values
```

```
    conference = rng.choice(
        ['BigTen', 'BigEast', 'WCC', 'Summit'],
        size=n,
        p=[0.20, 0.20, 0.25, 0.35]
    )

    bpr_means = {
        'BigTen': 12,
        'BigEast': 7,
        'WCC': 0,
        'Summit': -7
    }

    bpr = (
        np.array([bpr_means[c] for c in conference])
        + rng.normal(0, 6, size=n)
    )

    # Generate normally distributed errors
    error = rng.normal(0, sigma, size=n)

    # Generate log NIL using the specified linear model
    log_nil = (
        beta_0
        + beta_bpr * bpr
        + np.array([conference_effects[c] for c in conference])
        + error
    )

    return pl.DataFrame({
        'conference': conference,
        'bpr': bpr,
        'log_nil': log_nil,
        'nil_m': np.exp(log_nil)
    })

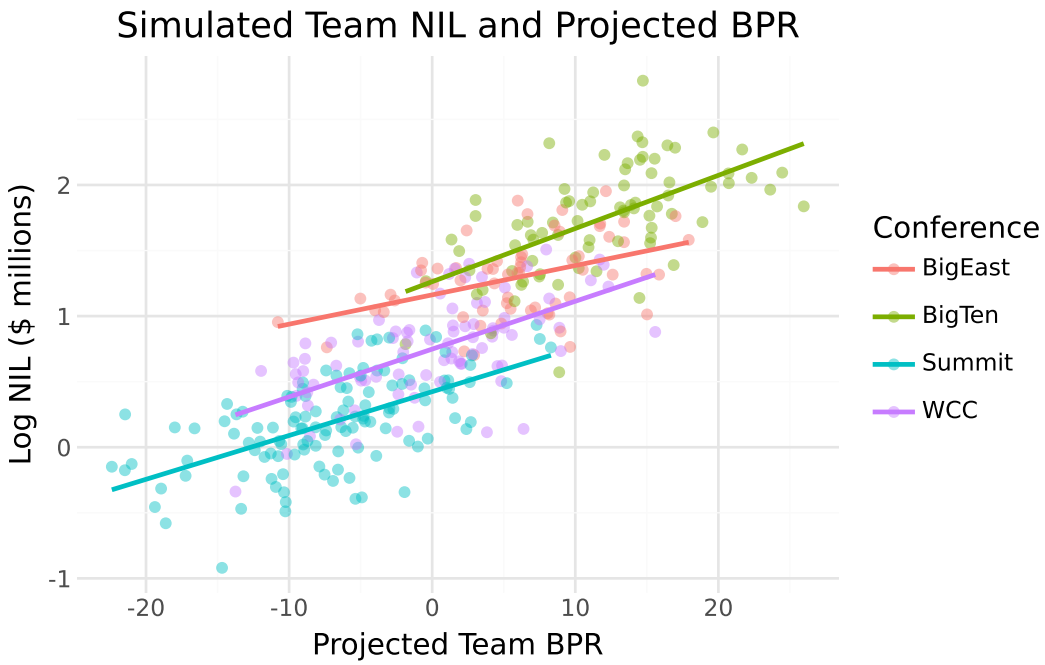
# Simulate 353 teams
sm_data = simulate_nil(
    n=353,
    beta_0=beta_0,
    beta_bpr=beta_bpr,
    conference_effects=conference_effects,
```

```
    sigma=sigma
)

sm_data.head()
```

conference str	bpr f64	log_nil f64	nil_m f64
”Summit”	-3.347431	0.585193	1.795338
”WCC”	-13.747737	-0.337965	0.71322
”Summit”	-5.1738	-0.003226	0.99678
”Summit”	-6.567799	-0.031759	0.96874
”BigTen”	14.483342	1.138393	3.121749

```
# Visualize the simulated relationship by conference
(ggplot(sm_data, aes(x='bpr', y='log_nil', color='conference'))
    + geom_point(alpha=0.45)
    + geom_smooth(method='lm', se=False)
    + labs(
        title='Simulated Team NIL and Projected BPR',
        x='Projected Team BPR',
        y='Log NIL ($ millions)',
        color='Conference'
    )
    + theme_minimal()
)
```




```
# Dummy code conference, using Big Ten as the reference
sm_train = (
    sm_data
    .to_dummies(columns=['conference'], drop_first=False)
    .select(pl.exclude('conference_BigTen'))
)

# Fit the same linear relationship used to generate the data
sim_fit = smf.ols(
    'log_nil ~ bpr + conference_BigEast + conference_WCC + conference_Summit',
    data=sm_train
).fit()

sim_fit.summary()
```

Dep. Variable:	log_nil	**R-squared:**	0.824
Model:	OLS	**Adj. R-squared:**	0.822
Method:	Least Squares	**F-statistic:**	406.7
Date:	Fri, 16 Oct 2026	**Prob (F-statistic):**	9.41e-130
Time:	15:27:38	**Log-Likelihood:**	-70.575
No. Observations:	353	**AIC:**	151.2
Df Residuals:	348	**BIC:**	170.5
Df Model:	4		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
Intercept	1.3408	0.045	29.732	0.000	1.252	1.430
bpr	0.0338	0.003	12.991	0.000	0.029	0.039
conference_BigEast	-0.2459	0.052	-4.692	0.000	-0.349	-0.143
conference_WCC	-0.5925	0.055	-10.721	0.000	-0.701	-0.484
conference_Summit	-0.9142	0.065	-14.136	0.000	-1.041	-0.787

Omnibus:	6.285	**Durbin-Watson:**	1.743
Prob(Omnibus):	0.043	**Jarque-Bera (JB):**	6.107
Skew:	-0.282	**Prob(JB):**	0.0472
Kurtosis:	3.313	**Cond. No.**	58.1

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
# Store the parameters used to generate the simulated data
true_parameters = {
    'Intercept': beta_0,
    'bpr': beta_bpr,
    'conference_BigEast': conference_effects['BigEast'],
    'conference_WCC': conference_effects['WCC'],
    'conference_Summit': conference_effects['Summit']
}

# Extract estimated coefficients and confidence intervals
sim_ci = sim_fit.conf_int()

recovery = pl.DataFrame({
    'parameter': list(true_parameters),
    'true_value': list(true_parameters.values()),
    'estimate': [
        sim_fit.params[p] for p in true_parameters
    ],
```

```
    'lower_95': [
        sim_ci.loc[p, 0] for p in true_parameters
    ],
    'upper_95': [
        sim_ci.loc[p, 1] for p in true_parameters
    ]
})

# Check whether the intervals contain the true parameters
recovery = recovery.with_columns(
    contains_true=pl.col('true_value').is_between(
        pl.col('lower_95'),
        pl.col('upper_95')
    )
)

recovery.with_columns(pl.col(pl.Float64).round(3))
```

parameter str	true_value f64	estimate f64	lower_95 f64	upper_95 f64	contains_true bool
”Intercept”	1.4	1.341	1.252	1.43	true
”bpr”	0.03	0.034	0.029	0.039	true
”conference_BigEast”	-0.25	-0.246	-0.349	-0.143	true
”conference_WCC”	-0.65	-0.593	-0.701	-0.484	true
”conference_Summit”	-1.0	-0.914	-1.041	-0.787	true

This simulation was done to look at whether OLS could recover known parameters under the data-generating process. We did 353 observations using the BPR coefficient, conference-specfice intercepts, and normally distributed error.

The OLS model recovered parameter estimates pretty close to the values used to generate the data. The estimated BPR coefficient was about 0.034 compared with the actual value of 0.30. The estimated conference effectes were also consistent with the relationships, and the 95% confidence intervals has the true parameters in this simulation

These results show that the estimations can recover the relationships when we assumed linear model correctly represents the data process. However, successful parameter recovery in simulated data does not establish that the same assumptions are valid for the observed NIL data.

Explore

Data Sources and Collection

This analysis uses two datasets:

The first dataset is from EvanMiya

The second data set is from The Sideline Men's Basketball NIL Tracker

Both datasets are downloaded as CSV files and were joined using school names. Since the tow sources has different names for some schools, they were standardized before merging.

Data Cleaning and Preparation

Load datasets

```
from pyprojroot.here import here

# Load the original 2026-27 team ratings and NIL estimates
team_data = pl.read_csv(
    here('data/evanmiya_team_ratings_2026_27.csv')
)

nil_data = pl.read_csv(
    here('data/sideline-basketball-nil-rankings.csv'),
    skip_rows=3
)

print("EvanMiya observations:", team_data.height)
print("Sideline observations:", nil_data.height)
print("EvanMiya columns:", team_data.columns)
print("Sideline columns:", nil_data.columns)
```

```
EvanMiya observations: 365
Sideline observations: 354
EvanMiya columns: ['season', 'rank', 'rank_excl_ineligible', 'team', 'obpr', 'dbpr', 'bpr',
Sideline columns: ['Rank', 'School', 'Conference', 'Tier', "Men's Basketball NIL Estimate ($
```

Standardize school names

```
# Standardize Sideline school names to match EvanMiya
school_names = {
  'American University': 'American', 'Detroit Mercy': 'Detroit', 'FIU': 'Florida Internationa
  'Grambling State': 'Grambling', 'IU Indianapolis': 'IU Indy', 'LIU': 'Long Island',
  'Louisiana': 'Louisiana-Lafayette', 'Louisiana Monroe': 'Louisiana-Monroe',
  'Loyola (Maryland)': 'Loyola Maryland', 'Maryland Eastern Shore': 'Maryland-Eastern Shore'
  'McNeese': 'McNeese State', 'Miami (FL)': 'Miami (Fla.)', 'NC A&T': 'North Carolina A&T',
  'NC Central': 'North Carolina Central', 'Nicholls': 'Nicholls State', 'Prairie View A&M':
  'Purdue Fort Wayne': 'Fort Wayne', 'Queens University': 'Queens', 'SE Missouri State': 'Sou
  'SIUE': 'SIU Edwardsville', 'Sam Houston': 'Sam Houston State', 'Seattle University': 'Seat
  'Southern Miss': 'Southern Mississippi', 'St. Thomas': 'St. Thomas (MN)', 'UConn': 'Connect
  'UIC': 'Illinois-Chicago', 'USC Upstate': 'South Carolina Upstate', 'UT Martin': 'Tennessee
  'UTRGV': 'Texas-Rio Grande Valley'
}
tier_names = {'Power 4': 'Power4', 'Lower-Tier': 'LowerTier'}


# Clean Sideline data and include conference
nil_clean = (
    nil_data
    .select(
        pl.col('School').alias('team'),
        pl.col('Tier').alias('tier'),
        pl.col('Conference').alias('conference'),
        pl.col("Men's Basketball NIL Estimate ($M)").alias('nil_m')
    )
    .with_columns(
        pl.col('team').replace(school_names),
        pl.col('tier').replace(tier_names),
        pl.col('conference').str.replace_all('[^A-Za-z0-9]', ''),
        pl.col('nil_m').cast(pl.Float64, strict=False)
    )
    .unique()
)


# Keep only the useful EvanMiya columns
team_clean = (team_data
  .select('team', 'rank', 'obpr', 'dbpr', 'bpr', 'tempo')
  .unique()
```

```
)

# Join teams with a NIL estimate and log-transform NIL
tm_data = (team_clean
  .join(nil_clean, on = 'team', how = 'inner')
  .with_columns(pl.col('nil_m').log().alias('log_nil'))
)
```

Join datasets

```
# Join EvanMiya ratings with Sideline NIL estimates
tm_data = (
    team_clean
    .join(nil_clean, on='team', how='inner', validate='1:1')
    .with_columns(
        pl.col('nil_m').log().alias('log_nil')
    )
    .sort('team')
)

# Verify the final columns
print(tm_data.columns)
print("Matched teams:", tm_data.height)
```

```
['team', 'rank', 'obpr', 'dbpr', 'bpr', 'tempo', 'tier', 'conference', 'nil_m', 'log_nil']
Matched teams: 353
```

Descriptive Statistics

Check missing data

```
# Teams in EvanMiya without a matching Sideline record
dropped = team_clean.join(
    nil_clean.select('team'),
    on='team',
    how='anti'
)
```

```
# Sideline teams without a matching EvanMiya record
unmatched_nil = nil_clean.join(
    team_clean.select('team'),
    on='team',
    how='anti'
)

# Summarize matching coverage
coverage = pl.DataFrame({
    'source': [
        'EvanMiya teams',
        'Sideline teams',
        'Matched teams',
        'Unmatched EvanMiya',
        'Unmatched Sideline'
    ],
    'teams': [
        team_clean.height,
        nil_clean.height,
        tm_data.height,
        dropped.height,
        unmatched_nil.height
    ]
})

print(coverage)

print("EvanMiya teams excluded:")
print(dropped.select('team', 'bpr').sort('bpr'))

print("Missing values in the matched data:")
print(tm_data.null_count())
```

```
shape: (5, 2)

 source              teams
 ---                 ---
 str                 i64

 EvanMiya teams      365
 Sideline teams      354
 Matched teams       353
```

```
  Unmatched EvanMiya   12
  Unmatched Sideline   1

EvanMiya teams excluded:
shape: (12, 2)

  team                    bpr
  ---                     ---
  str                     f64

  West Florida            -14.413759
  New Haven               -13.889283
  Le Moyne                -11.468939
  Mercyhurst              -10.674043
  West Georgia            -10.264072
  …                       …
  Missouri-Kansas City    -7.884104
  Navy                    -5.162617
  Arkansas-Little Rock    -4.794751
  Northern Kentucky       -4.283449
  College of Charleston   -1.164439

Missing values in the matched data:
shape: (1, 10)

  team   rank   obpr   dbpr   …   tier   conference   nil_m   log_nil
  ---    ---    ---    ---        ---    ---          ---     ---
  u32    u32    u32    u32        u32    u32          u32     u32

  0      0      0      0      …   0      0            0       0
```

Overall Descriptive Statistics

```python
# Describe the outcome, primary predictor, and sample
overall_summary = tm_data.select(
    pl.len().alias('teams'),
    pl.col('conference').n_unique().alias('conferences'),
    pl.col('nil_m').mean().alias('mean_nil_m'),
    pl.col('nil_m').median().alias('median_nil_m'),
    pl.col('nil_m').min().alias('min_nil_m'),
```

```
    pl.col('nil_m').max().alias('max_nil_m'),
    pl.col('bpr').mean().alias('mean_bpr'),
    pl.col('bpr').std().alias('sd_bpr'),
    pl.corr('bpr', 'log_nil').alias('cor_bpr_log_nil')
)

overall_summary.with_columns(
    pl.col(pl.Float64).round(3)
)
```

teams u32	conferences u32	mean_nil_m f64	median_nil_m f64	min_nil_m f64	max_nil_m f64	mean_bpr f64	sd_bpr f64	cor_ f64
353	32	2.8	1.15	0.3	15.8	0.289	10.379	0.81

By tier

```
# Summarize NIL and projected team strength by tier
tier_summary = (
    tm_data
    .group_by('tier')
    .agg(
        teams=pl.len(),
        mean_nil_m=pl.col('nil_m').mean(),
        median_nil_m=pl.col('nil_m').median(),
        mean_bpr=pl.col('bpr').mean(),
        sd_bpr=pl.col('bpr').std(),
        correlation=pl.corr('bpr', 'log_nil')
    )
    .sort('median_nil_m', descending=True)
)

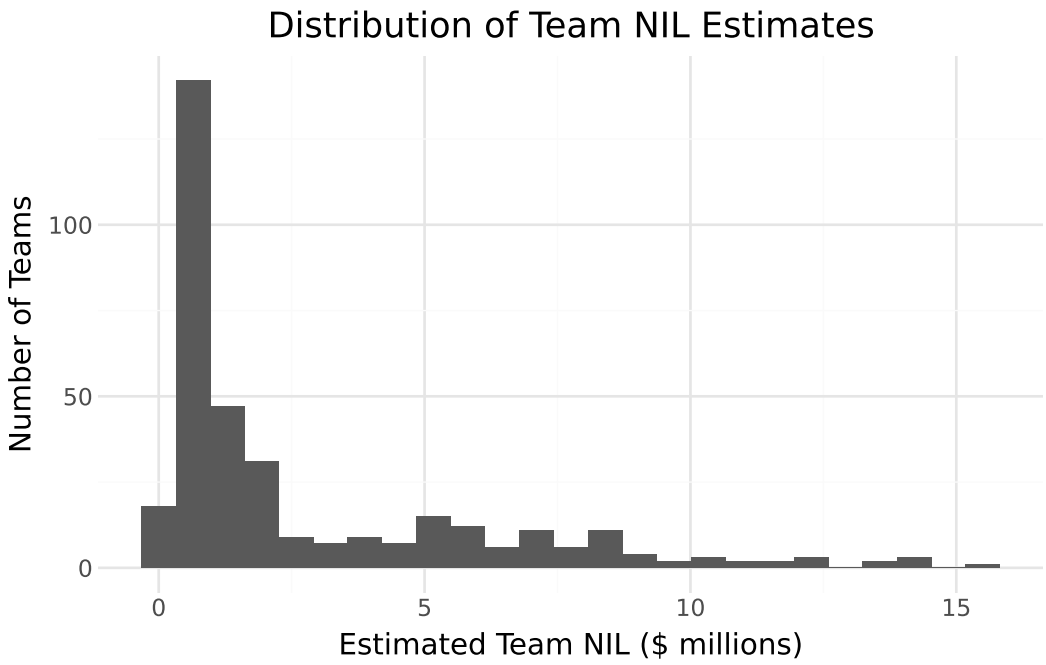
tier_summary.with_columns(
    pl.col(pl.Float64).round(3)
)
```

tier str	teams u32	mean_nil_m f64	median_nil_m f64	mean_bpr f64	sd_bpr f64	correlation f64
”Power4”	68	7.526	6.925	15.282	6.612	0.619
”G5”	53	2.362	1.9	0.674	7.009	0.56
”Basketball”	56	3.266	1.6	0.761	9.476	0.786
”LowerTier”	100	1.104	0.825	-4.714	5.177	0.552
”FCS”	68	0.807	0.75	-7.737	6.073	0.117
”Ivy”	8	0.413	0.4	-2.248	4.299	0.668

Exploratory Data Analysis

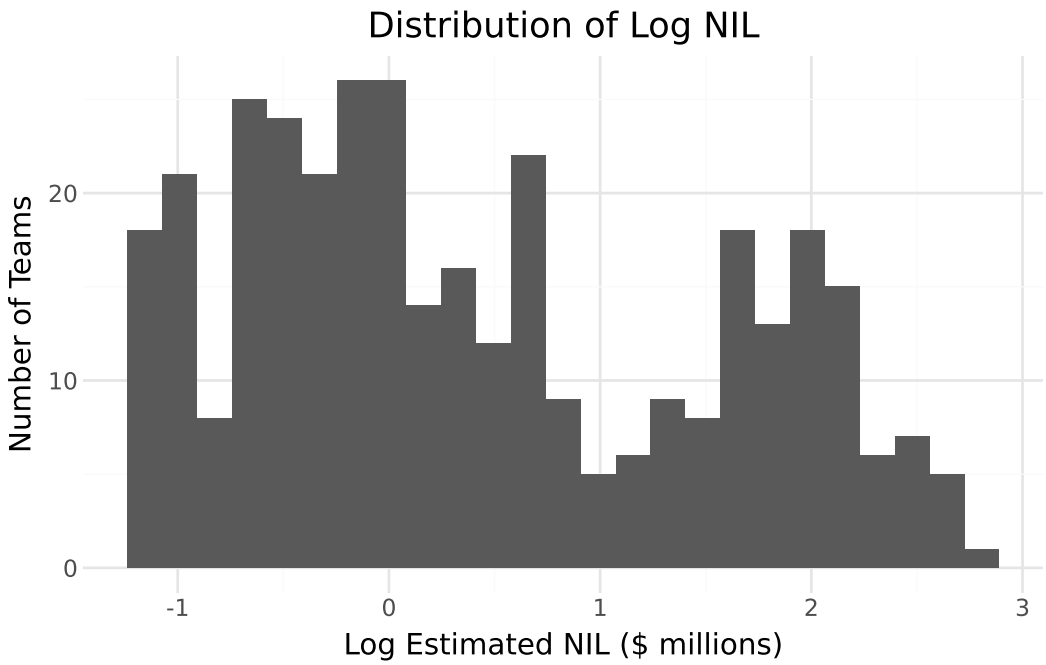
Viz 1: Estimated NIL distribution

```
(ggplot(tm_data, aes(x='nil_m'))
    + geom_histogram(bins=25)
    + labs(
        title='Distribution of Team NIL Estimates',
        x='Estimated Team NIL ($ millions)',
        y='Number of Teams'
    )
    + theme_minimal()
)
```



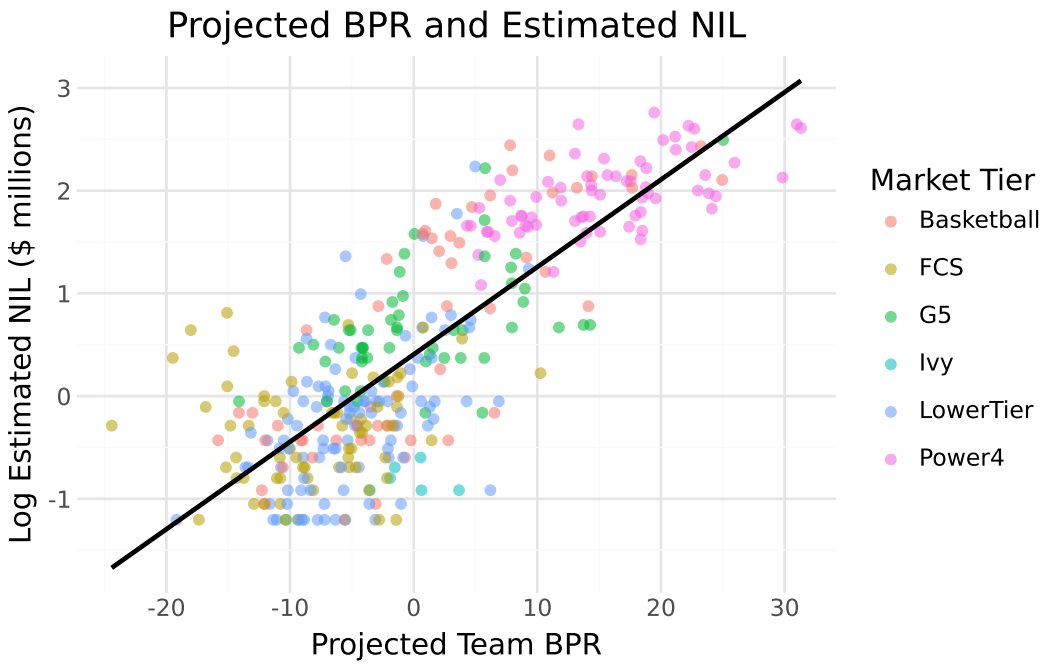
Viz 2: Log NIL

```
(ggplot(tm_data, aes(x='log_nil'))
    + geom_histogram(bins=25)
    + labs(
        title='Distribution of Log NIL',
        x='Log Estimated NIL ($ millions)',
        y='Number of Teams'
    )
    + theme_minimal()
)
```



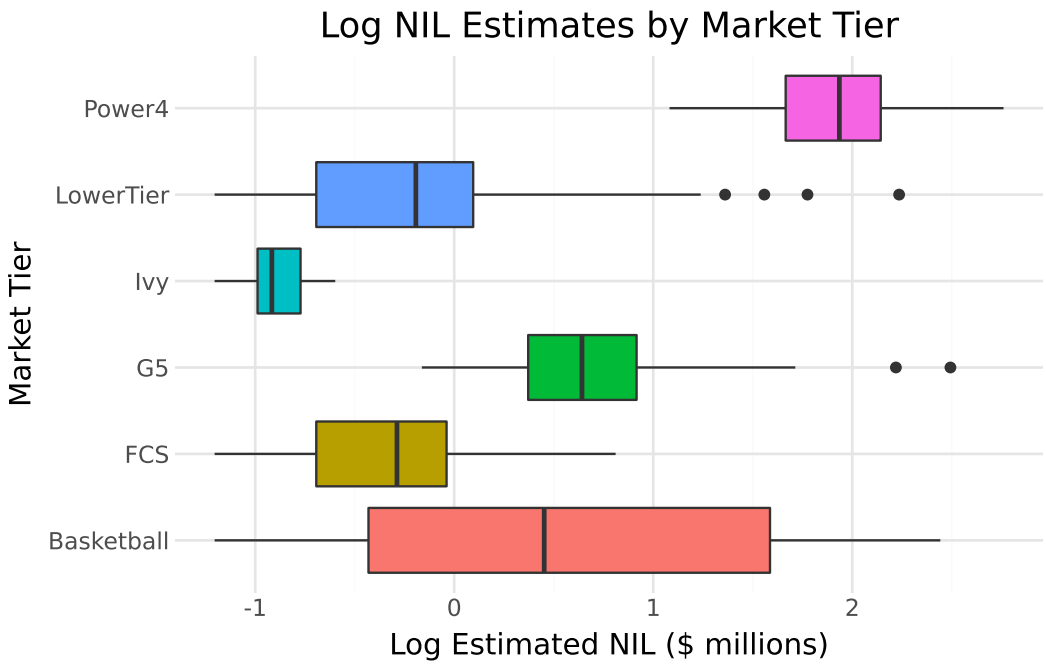
Viz 3: BPR and Log NIL

```
(ggplot(tm_data, aes(x='bpr', y='log_nil'))
    + geom_point(aes(color='tier'), alpha=0.55)
    + geom_smooth(
        method='lm',
        se=False,
        color='black'
    )
    + labs(
        title='Projected BPR and Estimated NIL',
        x='Projected Team BPR',
        y='Log Estimated NIL ($ millions)',
        color='Market Tier'
    )
    + theme_minimal()
)
```



Viz 4: NIL estimates by tier

```
(ggplot(tm_data, aes(x='tier', y='log_nil', fill='tier'))
    + geom_boxplot()
    + coord_flip()
    + labs(
        title='Log NIL Estimates by Market Tier',
        x='Market Tier',
        y='Log Estimated NIL ($ millions)'
    )
    + theme_minimal()
    + theme(legend_position='none')
)
```




```
# Final summary of the actual merged team data
eda_overall = tm_data.select(
    pl.len().alias('teams'),
    pl.col('conference').n_unique().alias('conferences'),
    pl.col('nil_m').mean().alias('mean_nil'),
    pl.col('nil_m').median().alias('median_nil'),
    pl.col('nil_m').min().alias('min_nil'),
    pl.col('nil_m').max().alias('max_nil'),
    pl.col('nil_m').skew().alias('nil_skew'),
    pl.col('log_nil').skew().alias('log_nil_skew'),
    pl.col('bpr').mean().alias('mean_bpr'),
    pl.col('bpr').std().alias('sd_bpr'),
    pl.corr('bpr', 'log_nil').alias('correlation'),
    pl.col('nil_m')
      .filter(pl.col('nil_m').is_between(0.2999, 0.3001))
      .count()
      .alias('teams_at_0_3m')
)

# Summary of team characteristics by market tier
eda_tiers = (
    tm_data
    .group_by('tier')
```

```
    .agg(
        teams=pl.len(),
        mean_nil=pl.col('nil_m').mean(),
        median_nil=pl.col('nil_m').median(),
        mean_bpr=pl.col('bpr').mean(),
        correlation=pl.corr('bpr', 'log_nil')
    )
    .sort('median_nil', descending=True)
)

# Compare matched and unmatched programs
unmatched = team_clean.join(
    nil_clean.select('team'),
    on='team',
    how='anti'
)

eda_coverage = pl.DataFrame({
    'group': ['Matched', 'Unmatched'],
    'teams': [tm_data.height, unmatched.height],
    'mean_bpr': [
        tm_data['bpr'].mean(),
        unmatched['bpr'].mean()
    ]
})

with pl.Config(tbl_cols=20, tbl_width_chars=180):
    print("\nOVERALL SUMMARY")
    print(eda_overall.with_columns(pl.col(pl.Float64).round(3)))

    print("\nTIER SUMMARY")
    print(eda_tiers.with_columns(pl.col(pl.Float64).round(3)))

    print("\nSAMPLE COVERAGE")
    print(eda_coverage.with_columns(pl.col(pl.Float64).round(3)))

    print("\nMISSING VALUES")
    print(tm_data.null_count())
```

```
OVERALL SUMMARY
shape: (1, 12)
```

teams	conferences	mean_nil	median_nil	min_nil	max_nil	nil_skew	log_nil_skew	mea
u32	u32	f64	f64	f64	f64	f64	f64	f64
353	32	2.8	1.15	0.3	15.8	1.648	0.379	0.2

TIER SUMMARY
shape: (6, 6)

tier	teams	mean_nil	median_nil	mean_bpr	correlation
str	u32	f64	f64	f64	f64
Power4	68	7.526	6.925	15.282	0.619
G5	53	2.362	1.9	0.674	0.56
Basketball	56	3.266	1.6	0.761	0.786
LowerTier	100	1.104	0.825	-4.714	0.552
FCS	68	0.807	0.75	-7.737	0.117
Ivy	8	0.413	0.4	-2.248	0.668

SAMPLE COVERAGE
shape: (2, 3)

group	teams	mean_bpr
str	i64	f64
Matched	353	0.289
Unmatched	12	-8.539

MISSING VALUES
shape: (1, 10)

team	rank	obpr	dbpr	bpr	tempo	tier	conference	nil_m	log_nil
u32	u32	u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	0	0	0	0	0	0	0

Data Limitations

We have a couple of limitations that affect the interpretation and gereralizability of our data

First, some of our NIL estimates from Sideline are a modeled values rather than the verified compensation. So our outcome should be interpreted as estimated NIL market and not the actual audit

Second, the analysis uses cross-sectional data froma single season. It doesn't capture how changes in NIL spending affect performance over time, and it doesn't account for other facotrs like program history, popularity, recruiting, donors, or how a player has a specific relationship with a college

Reconcile

Following our workflow, we start with a linear model predicting log NIL from BPR and market. We then look into whether conference give a better representation of differences between schools.

Our model diagnostics are conducted using our training data, while the held-out test data are kept for the predictive model evaluation. This divide reduces the risk of choosing a model based on its performance on data just to evaluate it

Validity and Representativeness

Validity

The validity of our analysis depends on whether our variables represent our objective. Our objective is to build NIL benchmarks associated with basketball strength and conference.

BPR gives us a quantitative measure of team performance, and Sideline's estimated NIL values give us a reference for financial resources. Both variables are relevant to our objective, although neither directly measures realized performance or verfied NIL value

Representativeness

We have 353 of the 365 teams from our datasets which 96.7% coverage.

The 12 unmatched programs have an average projected BPR of -8.539, compared with 0.289 among matched programs. This difference suggests that lower-performing teams are underrepresented in the analytical sample.

Our data will be examines using DFFITS. Big influence stats can identify teams that substantially affect fitted values, but those teams will not be removed just because they are unusual.

Linearity

80/20 Split

```
from sklearn.model_selection import train_test_split
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Sort teams for reproducibility
model_data = tm_data.sort('team')

# Reserve 20% of observations for final model evaluation
train_raw, test_raw = train_test_split(
    model_data,
    test_size=0.20,
    random_state=42
)

print("Training observations:", train_raw.height)
print("Testing observations:", test_raw.height)
print("Training conferences:", train_raw['conference'].n_unique())

# Verify both sets contain different teams
assert not (
    set(train_raw['team'].to_list())
    & set(test_raw['team'].to_list())
)
```

```
Training observations: 282
Testing observations: 71
Training conferences: 32
```

Fit the BPR and Tier model

```
# Create tier indicator variables (Power4 is the reference)
tier_train = (
    train_raw
    .to_dummies(columns=['tier'], drop_first=False)
    .select(pl.exclude('tier_Power4'))
)

tier_cols = sorted(
    c for c in tier_train.columns
    if c.startswith('tier_')
)

# Estimate log NIL from BPR and market tier
fit_tier = smf.ols(
    'log_nil ~ bpr + ' + ' + '.join(tier_cols),
    data=tier_train.to_pandas()
).fit()

# Attach residuals and fitted values in their original row order
tier_diag = tier_train.with_columns(
    pl.Series('resid_tier', fit_tier.resid.to_numpy()),
    pl.Series('fitted_tier', fit_tier.fittedvalues.to_numpy())
)

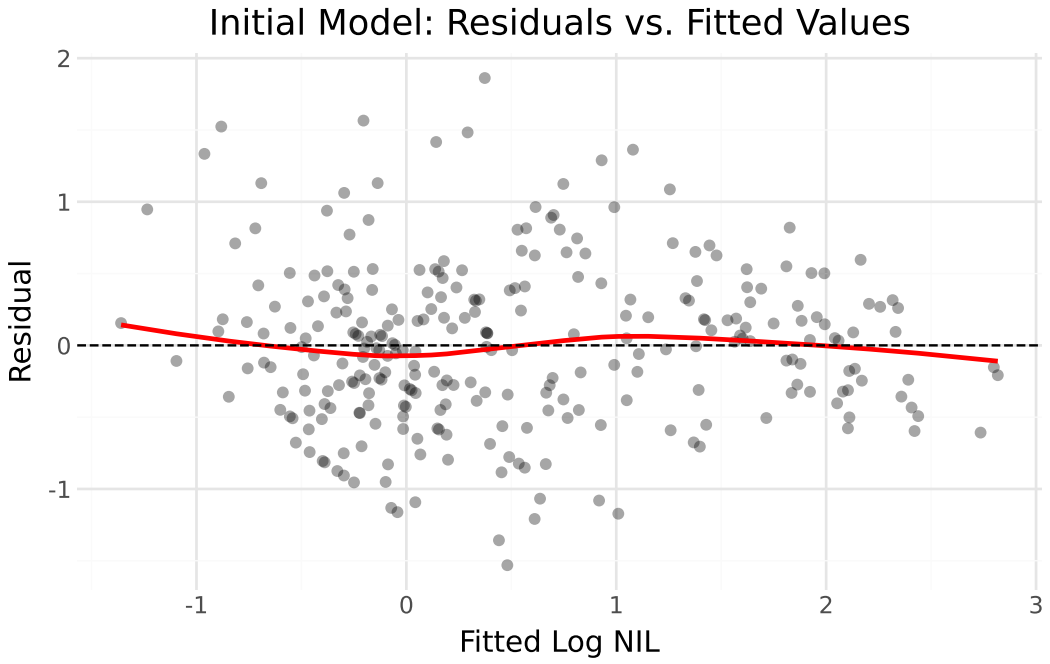
print("Initial R-squared:", round(fit_tier.rsquared, 3))
print("Initial adjusted R-squared:", round(fit_tier.rsquared_adj, 3))
print("BPR coefficient:", round(fit_tier.params['bpr'], 4))
```

```
Initial R-squared: 0.729
Initial adjusted R-squared: 0.723
BPR coefficient: 0.0551
```

Linearity Viz

```
(ggplot(tier_diag, aes(x='fitted_tier', y='resid_tier'))
    + geom_point(alpha=0.35)
    + geom_smooth(method='lowess', se=False, color='red')
```

```
    + geom_hline(yintercept=0, linetype='dashed')
    + labs(
        title='Initial Model: Residuals vs. Fitted Values',
        x='Fitted Log NIL',
        y='Residual'
    )
    + theme_minimal()
)
```



Independence

See the conference residuals

```
# Check whether conferences are nested within tiers
tier_nesting = (
    train_raw
    .group_by('conference')
    .agg(n_tiers=pl.col('tier').n_unique())
)
```

```
print(
    "Maximum tiers per conference:",
    tier_nesting['n_tiers'].max()
)

# Summarize residuals from the initial tier model by conference
conf_resid_initial = (
    tier_diag
    .group_by('conference')
    .agg(
        teams=pl.len(),
        mean_residual=pl.col('resid_tier').mean()
    )
    .sort('mean_residual')
)

# Display conferences with the largest negative and positive means
with pl.Config(tbl_rows=12):
    print(
        pl.concat([
            conf_resid_initial.head(5),
            conf_resid_initial.tail(5)
        ])
    )
```

```
Maximum tiers per conference: 1
shape: (10, 3)

 conference       teams   mean_residual
 ---              ---     ---
 str              u32     f64

 Patriot          3       -0.955064
 Southland        9       -0.632489
 MAAC             12      -0.499504
 MissouriValley   8       -0.448286
 WAC              7       -0.384914
 ConferenceUSA    10      0.33496
 BigEast          9       0.563489
 Atlantic10       13      0.56724
 WestCoast        9       0.87056
 MEAC             4       1.095628
```

Do model again with conference instead of tier

```
# Replace market tier with conference indicators
# Big Ten is the reference conference
cf_train = (
    train_raw
    .to_dummies(columns=['conference'], drop_first=False)
    .select(pl.exclude('conference_BigTen'))
)

conf_cols = sorted(
    c for c in cf_train.columns
    if c.startswith('conference_')
)

assert 'conference_BigTen' not in conf_cols

# Fit conference-adjusted OLS
fit_conf = smf.ols(
    'log_nil ~ bpr + ' + ' + '.join(conf_cols),
    data=cf_train.to_pandas()
).fit()


# Save residuals and fitted values, preserving conference names
cf_diag = (
    cf_train
    .with_columns(
        pl.Series('resid_conf', fit_conf.resid.to_numpy()),
        pl.Series('fitted_conf', fit_conf.fittedvalues.to_numpy())
    )
    .join(
        train_raw.select('team', 'conference'),
        on='team',
        how='left',
        validate='1:1'
    )
)
```

```
# Verify conference is included
print(cf_diag.columns)

# Compare in-sample model fit
fit_comparison = pl.DataFrame({
    'model': ['BPR + Tier', 'BPR + Conference'],
    'r_squared': [
        fit_tier.rsquared,
        fit_conf.rsquared
    ],
    'adjusted_r_squared': [
        fit_tier.rsquared_adj,
        fit_conf.rsquared_adj
    ],
    'residual_sd': [
        np.sqrt(fit_tier.mse_resid),
        np.sqrt(fit_conf.mse_resid)
    ]
})

print("Conference indicators:", len(conf_cols))
print(fit_comparison)
```

```
['team', 'rank', 'obpr', 'dbpr', 'bpr', 'tempo', 'tier', 'conference_ACC', 'conference_ASUN'
Conference indicators: 31
shape: (2, 4)

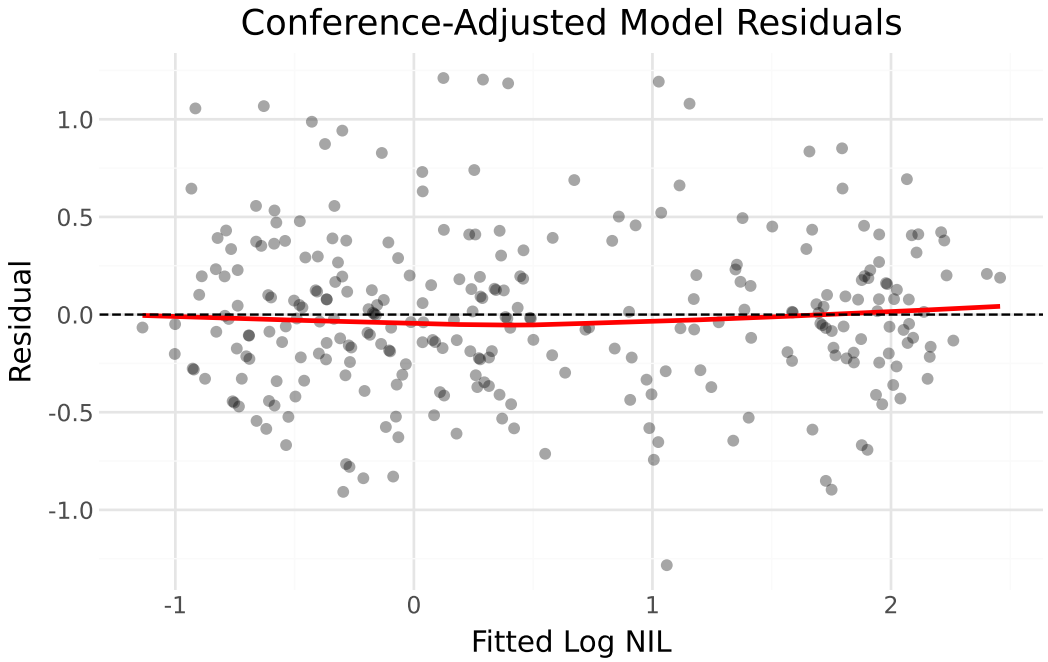
 model             r_squared   adjusted_r_squared   residual_sd
 ---               ---         ---                  ---
 str               f64         f64                  f64

 BPR + Tier        0.728621    0.7227               0.56723
 BPR + Conference  0.85623     0.837754             0.433882
```

Check model again rq

```
(ggplot(cf_diag, aes(x='fitted_conf', y='resid_conf'))
    + geom_point(alpha=0.35)
    + geom_smooth(method='lowess', se=False, color='red')
```

```
    + geom_hline(yintercept=0, linetype='dashed')
    + labs(
        title='Conference-Adjusted Model Residuals',
        x='Fitted Log NIL',
        y='Residual'
    )
    + theme_minimal()
)
```



The inital specfication models log NIL using BPR and market tier as the indicators. We then looked at conference residuals to see whether variation stayed within the market classifications

Because each conference belongs to a tier in the data, tier and conference indicators overlap information. That would be perfect multicollinearity.

So we replaced tier with conference. The new model has conference-specific intercepts while keeping a common BPR slope, with the Big Ten as our reference group

The adjustment is supposed to account for the fundamental difference in epxected NIL between conferences.

Constant Variance

Breusch-Pagan Test

```
# Test for nonconstant residual variance
bp_lm, bp_pvalue, bp_f, bp_f_pvalue = het_breuschpagan(
    fit_conf.resid,
    fit_conf.model.exog
)

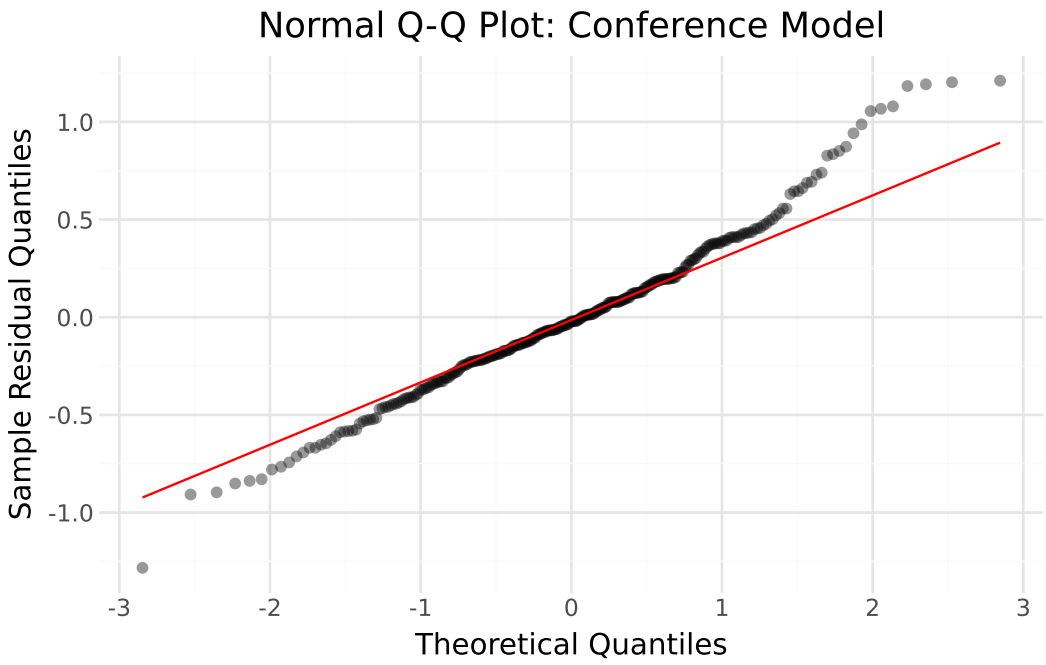
print("Breusch-Pagan LM statistic:", round(bp_lm, 3))
print("Breusch-Pagan p-value:", round(bp_pvalue, 4))
print("Breusch-Pagan F-test p-value:", round(bp_f_pvalue, 4))
```

```
Breusch-Pagan LM statistic: 55.917
Breusch-Pagan p-value: 0.0055
Breusch-Pagan F-test p-value: 0.0031
```

Normality

Q-Q Plot

```
(ggplot(cf_diag, aes(sample='resid_conf'))
    + stat_qq(alpha=0.4)
    + stat_qq_line(color='red')
    + labs(
        title='Normal Q-Q Plot: Conference Model',
        x='Theoretical Quantiles',
        y='Sample Residual Quantiles'
    )
    + theme_minimal()
)
```



Our Q-Q Plot points close to the reference line which suggests normality. We have a little tail in the end which could be a problem, but we should not automatically remove those teams

Look at large residuals

```
# Identify the largest positive and negative residuals
positive_residuals = (
    cf_diag
    .sort('resid_conf', descending=True)
    .select('team', 'conference', 'tier', 'nil_m', 'bpr', 'resid_conf')
    .head(5)
)

negative_residuals = (
    cf_diag
    .sort('resid_conf')
    .select('team', 'conference', 'tier', 'nil_m', 'bpr', 'resid_conf')
    .head(5)
)

print("Largest positive residuals:")
print(positive_residuals)
```

```
print("Largest negative residuals:")
print(negative_residuals)
```

```
Largest positive residuals:
shape: (5, 6)

 team           conference      tier         nil_m   bpr        resid_conf
 ---            ---             ---          ---     ---        ---
 str            str             str          f64     f64        f64

 Bradley        MissouriValley  Basketball   3.8     -2.1846    1.211271
 Drake          MissouriValley  Basketball   4.45    3.68482    1.203284
 Memphis        American        G5           9.2     5.775717   1.192691
 Massachusetts  MAC             G5           4.85    0.062609   1.183902
 Saint Mary's   WestCoast       LowerTier    9.35    4.977795   1.079856

Largest negative residuals:
shape: (5, 6)

 team              conference  tier         nil_m   bpr        resid_conf
 ---               ---         ---          ---     ---        ---
 str               str         str          f64     f64        f64

 Seattle           WestCoast   LowerTier    0.8     1.601404   -1.283236
 Presbyterian      BigSouth    FCS          0.3     -2.788702  -0.907735
 Butler            BigEast     Basketball   2.35    6.176608   -0.896627
 VCU               Atlantic10  Basketball   2.4     14.128465  -0.851398
 Central Arkansas  ASUN        LowerTier    0.35    -1.027155  -0.83797
```

Identifiability

Check design-matrix rank

```
# Compare design-matrix rank with the number of columns
design_matrix = fit_conf.model.exog

matrix_rank = np.linalg.matrix_rank(design_matrix)
n_parameters = design_matrix.shape[1]
```

```
print("Design-matrix rank:", matrix_rank)
print("Number of model parameters:", n_parameters)
print("Full column rank:", matrix_rank == n_parameters)
```

```
Design-matrix rank: 33
Number of model parameters: 33
Full column rank: True
```

Calculate Variance Inflation Factors

```
# Construct numerical predictor matrix including an intercept
vif_design = sm.add_constant(
    cf_train.select(['bpr'] + conf_cols).to_pandas(),
    has_constant='add'
).astype(float)

# Compute VIF for predictors, excluding the intercept
vif_table = pl.DataFrame({
    'predictor': list(vif_design.columns[1:]),
    'vif': [
        variance_inflation_factor(vif_design.values, i)
        for i in range(1, vif_design.shape[1])
    ]
}).sort('vif', descending=True)

print("Highest VIF values:")
print(vif_table.head(10))

print("Maximum VIF:", round(vif_table['vif'].max(), 3))
```

```
Highest VIF values:
shape: (10, 2)

 predictor               vif
 ---                     ---
 str                     f64

 bpr                     4.415567
 conference_MAAC         2.687855
```

```
  conference_SWAC          2.65073
  conference_SunBelt       2.263721
  conference_CAA           2.202082
  conference_AmericaEast   2.180744
  conference_Horizon       2.173925
  conference_MAC           2.165795
  conference_ASUN          2.150882
  conference_Southland     2.144534

Maximum VIF: 4.416
```

Model Refinements

Identify Influential Teams

```python
# Calculate influence diagnostics
influence = fit_conf.get_influence()
dffits_values = influence.dffits[0]

# Conventional screening threshold
n = train_raw.height
p = len(fit_conf.params)

dffits_cutoff = 2 * np.sqrt(p / n)

# Attach influence measures to the training observations
influence_data = cf_diag.with_columns(
    pl.Series('dffits', dffits_values),
    pl.Series('abs_dffits', np.abs(dffits_values))
)

influential_teams = (
    influence_data
    .filter(pl.col('abs_dffits') > dffits_cutoff)
    .sort('abs_dffits', descending=True)
    .select(
        'team', 'conference', 'nil_m',
        'bpr', 'resid_conf', 'dffits'
    )
)
```

```
print("DFFITS threshold:", round(dffits_cutoff, 3))
print("Potentially influential teams:", influential_teams.height)
print(influential_teams.head(10))
```

```
DFFITS threshold: 0.684
Potentially influential teams: 18
shape: (10, 6)
```

team	conference	nil_m	bpr	resid_conf	dffits
str	str	f64	f64	f64	f64
Memphis	American	9.2	5.775717	1.192691	1.239159
South Carolina Upstate	BigSouth	1.55	-14.566598	1.067376	1.182989
Drake	MissouriValley	4.45	3.68482	1.203284	1.154548
Bradley	MissouriValley	3.8	-2.1846	1.211271	1.148634
Seattle	WestCoast	0.8	1.601404	-1.283236	-1.129407
Colgate	Patriot	0.35	-3.064408	-0.523706	-1.049576
American	Patriot	1.0	-1.368151	0.478174	0.961646
Saint Mary's	WestCoast	9.35	4.977795	1.079856	0.953309
New Orleans	Southland	1.15	-8.632951	1.055634	0.932203
Presbyterian	BigSouth	0.3	-2.788702	-0.907735	-0.931222

Fit

Ordinary Least Squares

We used OLS because it estimates regression coefficients by minimizing the sum of squared residuals between oberved and predicted vales.

Our model is:

$$\log(Y_i) = \beta_0 + \beta_1 BPR_i + \sum_{c \neq \text{BigTen}} \gamma_c I(C_i = c) + \epsilon_i$$

Where Y_i is estimated team NIL in millions of dollars, β_1 represents the association between BPR and log NIL, and γ_c represents the difference associated with conference membership relative to the Big Ten.

```
# Reuse the conference-adjusted training model
fr_fit = fit_conf

# Report key OLS model statistics
ols_summary = pl.DataFrame({
    'statistic': [
        'Training observations',
        'R-squared',
        'Adjusted R-squared',
        'Residual standard error',
        'Model F-test p-value'
    ],
    'value': [
        fr_fit.nobs,
        fr_fit.rsquared,
        fr_fit.rsquared_adj,
        np.sqrt(fr_fit.mse_resid),
        fr_fit.f_pvalue
    ]
})

print(
    ols_summary.with_columns(
        pl.col('value').round(4)
    )
)
```

```
shape: (5, 2)

 statistic                value
 ---                      ---
 str                      f64

 Training observations    282.0
 R-squared                0.8562
 Adjusted R-squared       0.8378
 Residual standard error  0.4339
 Model F-test p-value     0.0
```

Frequentist Inference

Frequentist treats the parameters as fixed but unknown quatites. We quantify estimation ucertainty using standard errors, 95% confience intervals, and hypothesis tests.

For each coefficient, we test the null hypothesis that the parameter equals zero. A coefficient is statistically significant at the 5% level when its corresponding two-sided p-value is below 0.05, assuming the model and assumptions are met

A 95% confidence interval is constructed using a procedure that would contain the true population parameter in approximately 95% of repeated samples under the model assumptions. It is not a direct 95% probability statement about the fixed parameter.

Since the response variable is log NIL, coefficients are converted to percentage differences on the original NIL scale:

$$100(e^{\hat{\beta}} - 1)$$

This transformation gives an easier way to interpreted our results

Bayesian Inference

Evaluate

Model Fit and Interpretation

Model Comparison

Predictive Performance and Loss

Final Model Selection

Predict

Team NIL Benchmarks

Identifying Teams Above and Below Benchmark

Utah State Case Study

Communicate

Key Findings

Recommendations

Limitations and Uncertainty

Conclusion

References